LAB 6 · AN INTRODUCTION

Model Context Protocol

Tools stop belonging to one application.
Write the integration once; any client can use it.



FOR	Anyone who has written an integration twice.
READ IT	Before Sunday morning. Ten minutes.

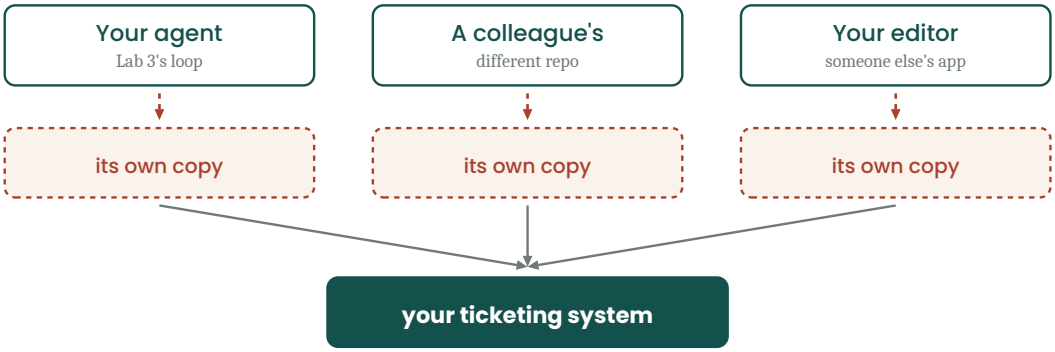
What it actually is

A standard way for a program that uses a model to discover and call tools that live somewhere else. A server offers capabilities; any client can consume them.

The reason it exists is easiest to see by describing the problem it removes. When you give an agent a tool, you write that tool into the agent. It works in that script and nowhere else. Want the same tool in a second application? Copy the code. A third? Copy it again.

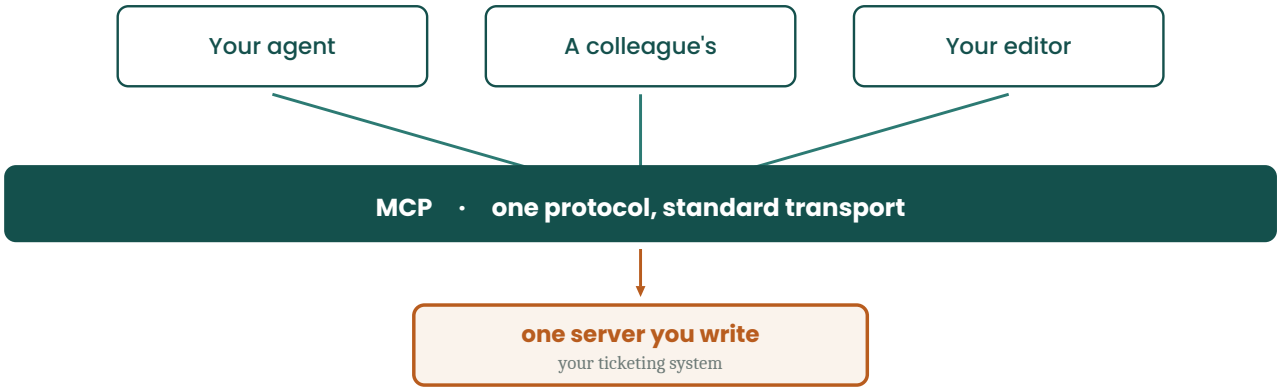
Everyone who has worked in infrastructure recognises this immediately. It is the problem that produced shared libraries, then internal APIs, then service discovery. The answer has always been the same: expose the capability once, behind an agreed interface, and let the callers find it.

Without a protocol



Three integrations to write, three to keep in step, three to fix when the API changes.

With one



Write it once. Every client that speaks the protocol gets it, including ones that do not exist yet.

> **THE PART THAT SURPRISES PEOPLE**
>
> The client does not know your server is a Python script talking to Postgres, and your server does not know or care which client is asking. Neither side was written with the other in mind. That is the whole point of a protocol, and it is why a tool you write today works in a client released next year.

Example 1 - Exposing something of your own

The case where you write the server.

Your company has a deployment system. It knows which version is in which environment, who released it, and what the rollback target is. Four people have written their own scripts to query it. Two of those scripts are broken and nobody has noticed.

Instead, one person writes a small MCP server with four narrow tools: `current_version`, `deploy_history`, `who_released`, `rollback_target`. Perhaps sixty lines. Then:

YOU	**What's in production right now, and who put it there?**
RUNS	`current_version(env="production")`
OUTPUT	`4.18.2, deployed 2026-11-06 14:21 UTC`
RUNS	`who_released(version="4.18.2")`
OUTPUT	`released by m.okafor, PR #3312`
AGENT	Production is on 4.18.2, released by Maryam Okafor yesterday afternoon from PR 3312. The previous version was 4.18.1 if you need a rollback target.

The useful part is not that one question got answered. It is that the same four tools now work in everyone's client, the team stopped maintaining four scripts, and the access rules live in one place where they can be reviewed.

Example 2 - Consuming somebody else's

The case where the work is already done.

You want your agent to read from a Postgres database, or search the web, or look at files on your machine. You could write those tools. Somebody already has, and theirs handles the connection pooling and the pagination and the error cases that yours would get wrong for the first month.

Adding one is a single command, and from that moment the tools simply exist in your client. You did not write a schema, a dispatch function or a loop.

> **WHAT TO NOTICE THE FIRST TIME**
>
> Your client will ask your permission before each call. Watch that prompt carefully, because it is the main security boundary in practice - and ask yourself honestly whether you would still be reading it on the two hundredth call of a long session.

The design decision that matters most

You will be tempted by one powerful tool. Resist it.

The deployment server above has four narrow tools. The obvious alternative is one general one - `run_query(sql)` - which is less code, more flexible, and handles questions you have not thought of yet.

It is also a genuinely bad idea, and the reason is worth stating precisely. The tool list is not a convenience for the model. **The tool list is the capability grant.** Four narrow tools give the model four things it can do. One SQL tool gives it everything the database connection can do, including every row of every table and every statement that is not a SELECT.

Four narrow tools	The model can ask four questions. You can audit which. You can require approval for one of them and not the others. A prompt injection in the data can, at worst, make it ask a question it was already allowed to ask.
One run_query tool	The model can do anything the credential can do. You cannot meaningfully approve a call without reading and understanding the SQL. "Is this statement destructive?" is not a question you can answer by inspecting a string.

> **THE SENTENCE TO REMEMBER**
>
> Narrow tools are a **permissioning mechanism**, not a limitation. Every tool you decline to write is a thing the model cannot be talked into doing.

Where else it goes wrong

The docstring is the interface.	Not documentation - interface. The model reads it to decide whether your tool is relevant and what to put in the arguments. A tool described as "Gets data" will be used wrongly or not at all, and nothing will error.
Server output enters your context.	Whatever a server returns is read by the model as part of the conversation. A server that returns text containing instructions is prompt injection with an extra hop - and if it is somebody else's server, it is somebody else's code deciding what your agent reads.
Third-party servers are dependencies.	Installing one is running someone's code with your credentials and your data. Read it, pin it, and apply whatever standard you would apply to any other dependency.

What to take into the lab

ONE

A tool is a contract: a name, a description, an argument shape. MCP is that contract with a standard transport, so it stops belonging to one application.

TWO

Narrow tools are a permissioning mechanism, not a limitation. You will want to write run_query. Do not write run_query.

THREE

The docstring is the interface. Write it for the model, because the model is the only thing that reads it.

In the lab you will connect to a server somebody else wrote, then write your own exposing a small database - about sixty lines - and register it. The experiment worth doing at the end is writing a tool with a deliberately vague docstring and watching the model fail to use it. Nothing makes the point faster.

The specification lives at modelcontextprotocol.io. It is readable in an afternoon, which is unusual for a specification and says something good about how much of this is genuinely new.